Visualising Developer Interactions in Code Reviews

Daniel Bee
Royal Holloway, University of London
Egham, UK
Daniel.Bee.2022@live.rhul.ac.uk

DongGyun Han
Royal Holloway, University of London
Egham, UK
DongGyun.Han@rhul.ac.uk

Abstract

Recent studies have demonstrated diverse benefits of code review. A common issue in the software development landscape is that developers struggle to balance a project's workload with other members of their team, resulting in burnout and inadequate code being pushed to production products. Consequently, the developers who are putting in the most work often do not get the credit, praise, and recognition that they deserve. This paper aims to outline how this operation can be assembled into an asynchronous, seamless visualisation that can be used to gauge the interactions between different developers in a given project. The development of this final product was split into three parts: a crawler, a migrator, and a visualiser. This visualisation technique will help companies improve their developers' workflows, teamwork, and time management when developing a project by analysing the interactions between team members. In addition to this, the visualisation allows users to select specific time frames for which they want to see interactions, and thus further enhancing its usability.

CCS Concepts

• **Software and its engineering → Maintaining software**.

Keywords

code review, visualisation, developer interactions

ACM Reference Format:
Daniel Bee and DongGyun Han. 2018. Visualising Developer Interactions in Code Reviews. In *Proceedings of IEEE/ACM International Conference on Software Engineering (ICSE '25).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Code review is a practice that all code changes are reviewed by one or more developers other than the change author themselves. Due to diverse benefits of code review, it has been widely adopted in both proprietary and open source projects. For example, previous studies have shown that it is useful for sharing knowledge and development context within a development team [1]. In addition, code review is helpful for improving software quality and design quality [5, 6]. Because of the diverse benefits, the practice has been widely adopted in both open and proprietary software projects. GitHub's pull request[1] is a representative example of code review.

One of the most challenging parts of code review is to find peers to review code changes. To mitigate the issue, many studies proposed reviewer recommendation techniques [7, 8]. These techniques recommend a group of developers for a given code review request. However, these techniques often limited to provide a guidance to individual code review requests rather than providing project wide guidances.

In this study, we propose a web service, `SERVICENAME`, that visualises developer interaction during pull request in GitHub. Based on a given project, our technique provides a visualisation of developers within a given timeframe. We expect the visualisation helps a software development team to understand the performance of their code review practices over time. They potentially identify a pair of developers who are struggling work each other (e.g., a pair of developers who rarely communicate each other). We also expect the visualisation help to plan how to pair developers to work together. They probably group the developers who actively communicate each other for boosting development speed. Alternatively, they can group the developers who have rarely worked together for leveraging fresh synergy between new combination.

`SERVICENAME` consists of three components: crawler, migrator, and visualiser. Since the visualisation requires multiple pull requests to comprehend the interactions, requesting such a large amount of data from GitHub could potentially be burdensome and slow. To avoid generating excessive traffic and response users request quickly, the crawler component retrieves pull request data from GitHub using the GitHub GraphQL API[2]. Once the crawling is done, the migrator component process the crawled data and store them into a database. Finally, the visualiser component visualises how developers interact to each other during code review.

ICSE '25, April 27–May 3, 2025, Ottawa, Ontario, Canada

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

[1] https://docs.github.com/en/pull-requests
[2] https://docs.github.com/en/graphql

2 Developer Interactions

```
# Pull requests must be provided
prs = []
# The timframe is optional parameters
timeframe = (start_date, end_date)

# The number of comments per developer.
nodes = {}
# The number of a developer pair within a PR.
links = {}

for pr in prs:
  if timeframe is given and pr is not within the
     timeframe:
    continue

  for comment in pr.comments:
    if pr.author == comment.author:
      continue

    nodes[comment.author] += 1

    author_pair = [pr.author, comment.author]
    author_pair.sort()
    links[author_pair] += 1
```

Listing 1: Algorithm to extract interactions

Listing 1 shows the algorithm to extract interactions from pull requests. When PRs are given, the algorithm iterates all PRs to extract information for the visualisation. If a timeframe is given to the algorithm, it only uses the PRs created within the timeframe. Otherwise, the algorithm leverages the given PRs.

For the PRs included in the scope of the visualisation (i.e., no timeframe is given or PRs created within the timeframe), the algorithm iterates all comments in them. If the PR author and comment author are same, the algorithm skip the comment. On the other hand, it increases the comment count (i.e., `nodes`) by one. This approach helps us to extract the contributions by each developer based on the number of comments on PRs that were not authored by themselves. The number of comments will be used to represent the size of each developer node in our visualisation. In addition, we construct a list that contains both PR and comment authors, then increase the count by one. To avoid unnecessary duplicates (e.g., {*JohnDoe*, *JaneDoe*} and {*JaneDoe*, *JohnDoe*}), we sort the list before increasing the count. The count of author pairs (i.e., `links`) will be used to control the thickness between developer nodes in the visualisation.

3 Visualisation Framework

The visualisation framework consists of three components as shown in Figure 1: Crawler, Migrator, and Visualiser. The crawler retrieves PRs and comments from GitHub. Then, migrator parse the data and store it to a database. Finally, a user can see a visualisation of a project via the web based visualiser. In this section, we explain the role of each component and implementation details. You can check the actual implementations on our replication package [3].

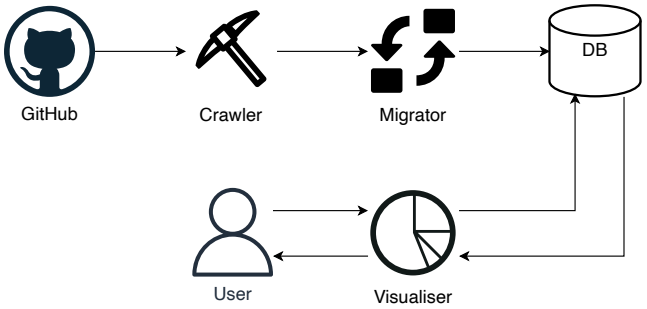

Figure 1: Overall framework

Table 1: Data Properties

Property	Description
`number`	Numeric id for pull request
`author.login`	Unique identifiers of developers
`author.avatarUrl`	URL of developer profile picture
`author.__typename`	User type (either human user or bot)
`createdAt`	Timestamp of PR/comment

3.1 Crawler

To visualise developer interactions, we need a large amount of pull request data. It will introduce huge traffics to code review platform if we request data whenever we need them. Such huge traffics potentially slow down the code review platform, so we do not query directly to the code review platform. Instead, we implemented a crawler that can crawl the latest code review data. The crawler leverages GitHub GraphQL API[4] and implemented in Python. We constructed a query that retrieves `author` and `createdAt` properties of PRs and PR comments. Table 1 explains the properties that we crawled.

3.2 Migrator

GraphQL is powerful approach handling large scale requests, but using GitHub GraphQL API directly for visualisation has three outstanding issues. First, GraphQL has additional types of nodes to optimise queries. For example, both PR and comment nodes should be surrounded by the `edges` and `node(s)` nodes which introduce unnecessary depth navigation. Second, GraphQL supports only limited conditional query. One of our visualiser's features supports visualising developer interactions within a specific time window. However, GraphQL does not support query that returns PRs that were authored within a specific date range. Last, traversing all PRs take time and require high computation power on the PR platform.

To mitigate the issues, we implemented a migrator that migrates the crawled API responses to a relational database instance. Figure 2 presents an entity relationship diagram of our database schema. The schema represents PRs and comments with the minimum subset of data that is required for visualisation. The migrator was also written in Python and we evaluated it with a MariaDB instance.

[3]TODO: We need to create a GitHub repo and put the implementations there
[4]https://docs.github.com/en/graphql

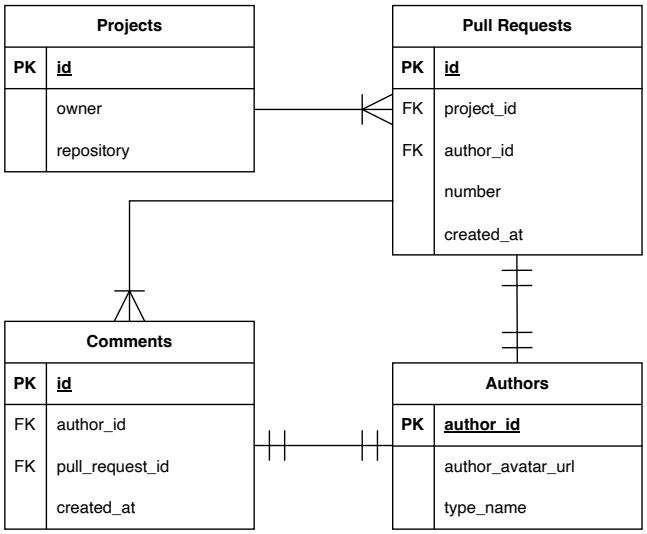

Figure 2: Entity Relationship Diagram

3.3 Visualiser

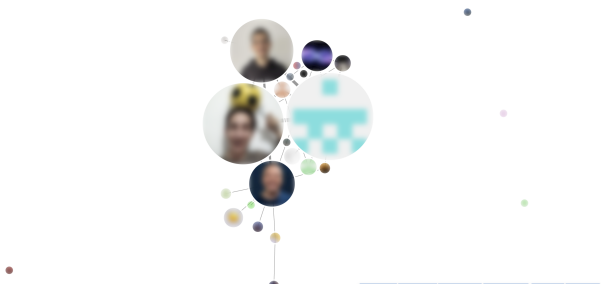

Figure 3: An example of visualisation. We blurred the actual profile photos to protect developers' privacy

The visualiser is responsible for visualising developer interactions. Users can select a specific project and time window to visualise developer interactions. For example, a user can request the visualiser to visualise developer interactions in the TypeScript project for a week from 3rd June 2024.

We implemented the visualiser as a web service that consists of two main components: backend and frontend. The backend component was implemented by using Java and Spring Framework. It retrieves data from the database upon request and run the algorithm described in Section 2. The results are serialised into a JSON string and return to the frontend. The frontend component is implemented by using TypeScript and React Framework. Since developer interactions can be visualised as a graph, we adopted the Force-directed graph component[5] in D3.

Figure 3 shows an example of the visualisation. Based on the algorithm presented in Section 2, the size of each node represents the number of comments written by each developer with their avatar image on GitHub. Please note that we blurred images on the paper, but real implementation shows clear images. The thickness of edges between nodes denote the number of comments shared by the two developers. To handle extremely large or small cases, we use normalised values for both nodes and edges.

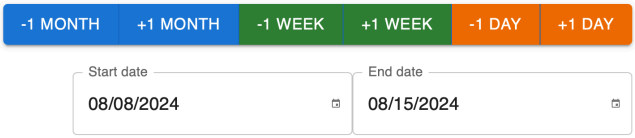

Figure 4: Date Controller

On the right bottom of the frontend, we put a date controller as shown in Figure 4. The user can select specific date by clicking the date field. Once they select a pair of dates, they can easily update the date scope by clicking the buttons above. Whenever the dates updated, the graph visualisation is also dynamically updated.

4 Related Work

When working in a development team for any kind of project, curating the best and most efficient workflow for team members is paramount. Even with the recent disruption caused by the COVID-19 pandemic, there is little excuse for teams to not be working at a high productivity. This is reflected when Russo et al. said: "Results suggest that the time software engineers spent doing specific activities from home was similar when working in the office"[3]. A visualisation can offer several benefits in clearly identifying the strengths and weaknesses in a team's interactions, dynamics, and workloads. A development team is effectively a microcosm of a social network, and graph structures have been great at encapsulating these as said by Majeed and Rauf: "Recently, graphs have been extensively used in social networks (SNs) for many purposes related to modelling and analysis of the SN structures, SN operation modelling, SN user analysis, and many other related aspects"[4]. Using data from GraphQL was useful in this decision since it was easy to crawl this data to scale if workloads are too one-sided, or if two team members are taking up too much of the work. Companies often struggle to find the right balance between allowing their developers to shine and burning them out, and it is even harder to figure out which one is taking place in a particular team. Having each node and link between nodes in the visualisation be scaled in size dependent on the crawled data provides a normalised dataset that is straightforward to track and monitor throughout development.

In addition to this, the physics of the nodes within the graph mean that nodes that are connected often surround each other, especially if one node has several children. This behaviour is useful for noticing patterns of dependence in a team and results in early problems being resolved promptly without hindering the rest of the development cycle. On the other hand, if there are any nodes that have no links between any other developers, then they can be identified as someone not partaking appropriately in a teamwork environment. This can then be addressed by the team manager. Multiple projects are supported by the system by default (as a result of rigid database design), but it is challenging to crawl, split, and migrate all the necessary data manually.

[5]https://observablehq.com/@d3/force-directed-graph/2

However, the visualisation that is created is only best applied in larger projects where more nodes and links are visible. Smaller projects will not be able to benefit from the graph analysis since they will most likely be aware of any team issues already that are simply consolidated in the visualisation.

Lastly, it is important to consider the reasons for why a project may have inefficient workflow in their teams. For example, negative feedback from peers may result in developers in abandoning projects after only a few code reviews, leaving other teammates to pick up the pieces as Egelman et al. wrote: "In the process, developers may have negative interpersonal interactions with their peers, which can lead to frustration and stress; these negative interactions may ultimately result in developers abandoning projects"[2].

5 Conclusion

Visualising developer interactions in code reviews is a vital part of the development process to ensure efficient and healthy progress for a project. Through easy graph analysis, one can discover team dynamics and relationships that showcase the weaknesses in a team's workflows that can then be addressed by appropriate parties in a short timeframe.

In this paper, the several components that make up the efficient web system that was built to power and support this visualisation will be broken down. Several technologies will be evaluated and explored that made the development of this project a lot faster than it otherwise would have been. The purpose and related work of the system will also be discussed.

As future work, the plan is to make the system more automated and reduce the manual labour for the end-user during the crawling, splitting, and migration stages of the data collection phase. Graph propagation is another feature in plans to allow individual users to be the root of the graph and propagate to their contributor counterparts. Extension features also include node colour coding and project mixing and matching (i.e. multiple projects in a singular visualisation).

References

[1] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*.

[2] Carolyn D. Egelman et al. 2020. Predicting Developers' Negative Feelings about Code Review. In *2020 IEEE/ACM 42nd International Conference on Software Engineering*. 174–185.

[3] Daniel Russo et al. 2021. The Daily Life of Software Engineers during the COVID-19 Pandemic. In *43rd International Conference on Software Engineering: Software Engineering in Practice*. IEEE, 364.

[4] Abdul Majeed and Ibtsisam Rauf. 2020. Graph Theory: A Comprehensive Survey about Graph Theory Applications in Computer Science and Social Networks. In *School of Information and Electronics Engineering*. Korea Aerospace University.

[5] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the Working Conference on Mining Software Repositories (MSR)*.

[6] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do Code Review Practices Impact Design Quality? A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.

[7] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for Modern Code Review. In *IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.

[8] Yue Yu, Huaimin Wang, Gang Yin, and Tao Wang. 2016. Reviewer recommendation for pull-requests in GitHub: What can we learn from code review and bug assignment? *Information and Software Technology* (2016).